

SIMATS ENGINEERING

ASSESSMENT TOOL 2

Case study

COURSE NAME: Computer Networks

COURSE CODE: CSA07

COURSE OUTCOME COVERED

CO3: *Analyze the performance of core network protocols such as TCP, UDP, HTTP, and DNS under varying conditions*

Assessment Tool Weightage: 40%

Case study

QUESTIONS:4

Total Marks:40

1.A multinational IT company uses a cloud-based video conferencing platform for daily client meetings. During peak business hours, employees experience frequent voice distortion, frozen video frames, and delayed communication. Network monitoring reports indicate an **8% packet loss**, **180 ms jitter**, and fluctuating bandwidth utilization. The IT administrator suspects that the issue is related to the transport protocol or application-level buffering strategy.

Questions

1. Analyze whether the problem is primarily caused by the **transport protocol (TCP/UDP)** or the **application layer**.
2. Justify your answer based on the communication requirements of real-time multimedia applications.
3. Recommend suitable protocol-level or application-level improvements to enhance conferencing quality.

2.A multinational software company hosts its web services across multiple continents. Employees frequently report long delays before the application homepage loads. Network analysis shows that the **average DNS lookup time is approximately 800 ms**, significantly affecting user experience. The organization plans to redesign its DNS infrastructure while ensuring continuous service availability during server failures.

Questions

1. Analyze the reasons behind the high DNS resolution time.
2. Propose a suitable DNS architecture using techniques such as **Anycast DNS**, **DNS pre-fetching**, and **TTL optimization** to reduce the lookup time below **50 ms**.
3. Evaluate the advantages and limitations of your proposed architecture in terms of performance, scalability, and availability.

3.A cloud service provider delivers applications to millions of users worldwide. During promotional events, users experience intermittent delays in accessing cloud-hosted services. Investigation reveals that DNS queries are taking nearly **800 ms** because all requests are directed to a single primary DNS server. The company wants a highly available DNS solution capable of handling global traffic efficiently.

Questions

- 4. Examine the impact of centralized DNS architecture on application performance.
- 5. Design a distributed DNS solution that minimizes lookup latency while maintaining high availability.
- 6. Analyze how **TTL values**, **Anycast routing**, and **DNS caching** influence system performance and fault tolerance.

4.A financial institution operates an electronic stock trading platform where thousands of buy and sell orders are submitted every second. During market opening hours, the **average order acknowledgment latency increases from 1 ms to 40 ms**, resulting in delayed trade confirmations and customer dissatisfaction. Network engineers observe increased TCP retransmissions, longer connection establishment times, and excessive buffering.

Questions

- 1. Analyze three possible TCP-related factors responsible for the latency increase, considering **flow control**, **Nagle's algorithm**, and **SYN queue limitations**.
- 2. Explain how each factor affects transaction processing in high-frequency systems
- 3. Recommend appropriate TCP configuration changes to reduce latency and improve transaction reliability.

RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Case	25	Thorough understanding; clearly identifies all key issues	Good understanding with most issues identified	Basic understanding; some issues missed	Poor understanding; problem not identified correctly
Application of Concepts	25	Effectively applies relevant concepts to analyze the case deeply	Applies appropriate concepts with minor gaps	Limited application of concepts	Fails to apply relevant concepts
Analysis	20	In-depth analysis with strong reasoning and insights	Good analysis with logical reasoning	Basic analysis with limited depth	Weak or no analysis; lacks reasoning
Solution Design	20	Innovative, feasible solutions with strong justification	Logical solutions with adequate justification	Basic solutions with weak justification	Incorrect or no solution provided
Clarity	10	Clear, well-structured, and easy to understand	Organized and understandable presentation	Limited clarity and structure	Poor clarity; difficult to understand

SIMATS ENGINEERING

ASSESSMENT TOOL 2 - Case Study

COURSE NAME: Computer Networks

COURSE CODE: CSA07

Case Study 1

1. Analyze whether the problem is primarily caused by the transport protocol (TCP/UDP) or the application layer.

The symptoms described — voice distortion, frozen video frames, delayed communication, 8% packet loss, and 180 ms jitter — point primarily to the transport protocol layer, specifically to the use (or misuse) of TCP for real-time media. If the video conferencing platform is using TCP for audio/video streams, TCP's built-in reliability mechanisms (retransmission of lost segments and in-order delivery) would force the receiver to wait for retransmitted packets before playing out any data. This directly explains the frozen frames and delayed communication. The 8% packet loss and high jitter are network-level symptoms, but the way they manifest as user-visible problems is determined by the transport protocol's behavior. If UDP were being used instead, occasional packet loss would cause brief glitches rather than freezing, since UDP does not wait for retransmissions. Application-layer buffering strategy is a secondary contributing factor (e.g., a jitter buffer that is too large or too small), but the core issue is the choice/behavior of the transport protocol under lossy, jittery network conditions.

2. Justify your answer based on the communication requirements of real-time multimedia applications.

Real-time multimedia applications such as video conferencing prioritize timeliness over completeness. A video or audio frame that arrives late is effectively useless, because it cannot be inserted back into a live stream without causing a visible or audible stall. TCP is designed for the opposite priority: it guarantees complete, ordered, error-free delivery, even if that means introducing arbitrary delay through retransmissions and congestion control back-offs. When TCP encounters packet loss, it reduces its sending rate (congestion avoidance) and retransmits lost segments, which increases latency and jitter, and any application waiting for in-order data will stall until the missing segment arrives. UDP, in contrast, has no retransmission or ordering guarantees, so the application receives data as soon as it arrives and can simply discard or conceal missing packets. This is why virtually all real-time voice/video systems (e.g., WebRTC-based platforms) use UDP-based transport, allowing minor loss to be tolerated for the sake of low, consistent latency — which is exactly the trade-off this case study needs.

3. Recommend suitable protocol-level or application-level improvements to enhance conferencing quality.

- i. Switch the media transport to UDP-based protocols (e.g., RTP/RTCP over UDP, or WebRTC) instead of TCP, so late data is simply skipped rather than causing stalls.
- ii. Use RTCP feedback and adaptive bitrate/jitter buffer algorithms that dynamically adjust buffer size and encoding rate based on measured jitter and loss, rather than using a fixed buffer.
- iii. Apply Forward Error Correction (FEC) or Packet Loss Concealment (PLC) so that a lost packet can be reconstructed or masked instead of degrading quality.
- iv. Implement Quality of Service (QoS) marking (DSCP) on the network to prioritize real-time voice/video traffic over best-effort traffic during peak hours.

- v. Use congestion-control mechanisms designed for real-time media, such as Google Congestion Control (GCC), to adapt sending rate smoothly without the harsh cutbacks of TCP-style congestion control.

Case Study 2

1. Analyze the reasons behind the high DNS resolution time.

An average DNS lookup time of 800 ms is far higher than the typical 20–50 ms expected for a well-designed system, and points to several likely causes. First, the DNS infrastructure may rely on a small number of centralized name servers that are geographically distant from many users, so every query has to travel a long round-trip path regardless of where the user is located. Second, low or poorly tuned Time-To-Live (TTL) values on DNS records would force resolvers to repeatedly query authoritative servers instead of serving cached answers, adding latency on nearly every request. Third, the resolution chain itself may be too long — if a query has to walk through multiple recursive resolvers, and then multiple levels of authoritative servers (root, TLD, and finally the domain's own authoritative server) without effective caching at intermediate resolvers, the cumulative round-trip delay adds up quickly. Finally, a lack of geographically distributed DNS servers means there is no way to route a user's query to a nearby, low-latency server, which is essential for a service used across multiple continents.

2. Propose a suitable DNS architecture using techniques such as Anycast DNS, DNS pre-fetching, and TTL optimization to reduce the lookup time below 50 ms.

A redesigned architecture should combine the following techniques:

- i. Anycast DNS: Deploy the same DNS server IP address at multiple geographically distributed points of presence (PoPs) across continents. Because Anycast relies on standard routing protocols to send a query to the topologically nearest server advertising that address, users are automatically directed to a nearby server, sharply cutting round-trip time, and if one PoP fails, traffic is automatically rerouted to the next-nearest one.
- ii. TTL Optimization: Set TTL values for DNS records to a value that balances caching benefit against the need for updates (e.g., 300 seconds for frequently-changing records, higher for stable ones). This lets recursive resolvers and client-side caches serve most repeat queries locally, avoiding a fresh round-trip to authoritative servers for every request.
- iii. DNS Pre-fetching: Have the client browser or application resolve DNS names for likely-needed resources in advance (e.g., while a page is loading, or during idle time), so that by the time an actual connection is needed, the IP address is already cached and available instantly.
- iv. Additionally, deploying redundant authoritative name servers in multiple regions (secondary DNS providers) and using recursive resolvers with large local caches close to end users further reduces both average latency and single points of failure.

3. Evaluate the advantages and limitations of your proposed architecture in terms of performance, scalability, and availability.

Performance: Anycast DNS combined with TTL optimization and pre-fetching should bring lookup times well under 50 ms for most users, since queries are answered by a nearby server or served directly from cache rather than travelling across continents. **Scalability:** Because Anycast allows any number of PoPs to answer using the same IP address, the architecture scales horizontally simply by adding more server locations, and load is naturally distributed by network topology rather than a single server having to handle all global traffic. **Availability:** Anycast provides automatic failover — if one server/PoP goes down, routing protocols redirect queries to the next nearest healthy server without requiring any client-side reconfiguration, and using multiple independent secondary DNS providers avoids a single point of failure. **Limitations:** Anycast routing depends on the underlying network's BGP behavior, which can occasionally route users to a suboptimal or overloaded server (imperfect

route convergence), and debugging Anycast issues is harder since the same IP can correspond to different physical servers depending on the requester's location. Longer TTLs improve performance but slow down the propagation of DNS changes (e.g., during failover or record updates), so there is a trade-off between caching efficiency and update responsiveness that must be tuned carefully.

Case Study 3

4. Examine the impact of centralized DNS architecture on application performance.

A centralized DNS architecture, where all queries worldwide are directed to a single primary DNS server, creates several performance problems that become especially visible under load, as seen in this case (queries taking nearly 800 ms during promotional events). First, every user's query must physically travel to wherever that single server is located, meaning users far from the server experience high propagation delay purely due to distance, regardless of server capacity. Second, a single server has a finite processing capacity; when traffic surges during high-demand events, the query queue backs up, and response times degrade sharply as the server becomes a bottleneck. Third, a centralized design creates a single point of failure — if that server becomes overloaded or goes offline, DNS resolution fails for all users globally, not just a subset, since there is no alternate path for queries to take. Overall, this architecture does not scale with global user growth or traffic spikes, and directly translates into poor application performance, since virtually every request to any hosted service must first pass through this slow, congested resolution step before any actual application data can be requested.

5. Design a distributed DNS solution that minimizes lookup latency while maintaining high availability.

The proposed solution replaces the single primary server with a distributed, redundant DNS layer built around the following components:

- i. Anycast-based DNS server deployment: Multiple DNS servers hosted at geographically dispersed data centers, all advertising the same IP address via Anycast, so that BGP routing automatically sends each user's query to the nearest available server.
- ii. Secondary/redundant authoritative servers: Deploy DNS service across at least two independent providers or clusters, so a failure or attack on one does not take down resolution entirely.
- iii. Hierarchical caching resolvers: Place recursive resolvers with substantial caches at regional or ISP level, so repeat queries for popular domains are answered locally without reaching the authoritative layer at all.
- iv. Load-balanced query handling: Within each regional cluster, use multiple servers behind a load balancer or health-checked cluster so traffic spikes at any single location are absorbed without degrading response time.
- v. Health monitoring and automatic failover: Continuously monitor server health at each PoP and automatically withdraw a failed node's route advertisement, so Anycast routing seamlessly redirects its traffic elsewhere.

6. Analyze how TTL values, Anycast routing, and DNS caching influence system performance and fault tolerance.

TTL values: A shorter TTL means DNS records expire from caches quickly, forcing more frequent queries back to the authoritative server — this increases average latency but allows changes (like failover to a backup IP) to propagate faster. A longer TTL reduces load on authoritative servers and speeds up repeat lookups by serving cached answers, but slows down how quickly the system can respond to a failure, since stale cached records may keep pointing users to a down server until the TTL expires. A well-tuned architecture uses moderate TTLs for stable services and shorter TTLs for records that may need to fail over quickly.

Anycast routing: By allowing many physically distinct servers to share one IP address, Anycast directly reduces lookup latency (queries go to the nearest server) and directly improves fault tolerance (if a server or its route disappears, routers automatically converge on the next-nearest advertiser without any client-side change required). Its main limitation is that failover speed depends on how quickly routing protocols converge, which is usually fast but not instantaneous.

DNS caching: Caching at multiple levels — client, recursive resolver, and ISP resolver — means that a large fraction of queries are answered without ever reaching the authoritative servers at all. This dramatically reduces average latency system-wide and reduces load on the authoritative layer, indirectly improving fault tolerance by making the system resilient even if the authoritative servers are briefly unreachable, since many clients can continue operating from cached answers until the TTL expires.

Case Study 4

1. Analyze three possible TCP-related factors responsible for the latency increase, considering flow control, Nagle's algorithm, and SYN queue limitations.

- i. Flow Control: TCP uses a receive window (advertised by the receiver) to prevent the sender from overwhelming it. Under heavy load, if the receiving application (or OS buffer) cannot drain incoming data fast enough, the advertised window shrinks, forcing the sender to pause or slow down transmission even though the network itself may have capacity — this directly adds latency to every subsequent segment.
- ii. Nagle's Algorithm: This algorithm delays sending small TCP segments, buffering them together into fewer, larger packets to improve network efficiency for bulk transfers. For a trading system sending many small, latency-sensitive messages (like individual order acknowledgments), Nagle's algorithm can hold a small packet back waiting for either more data to batch or an ACK for previously sent data, directly adding tens of milliseconds of unwanted delay to each message.
- iii. SYN Queue Limitations: Each listening TCP socket maintains a fixed-size queue for incoming half-open connections (SYN received, waiting for the final ACK of the three-way handshake). During market opening, a burst of simultaneous new connection attempts can exceed this queue's capacity, causing new SYN packets to be dropped or delayed until space frees up, which increases connection establishment time and, by extension, the time before an order can even be submitted.

2. Explain how each factor affects transaction processing in high-frequency systems.

In a high-frequency trading environment, thousands of orders are submitted and acknowledged every second, so even small per-message delays compound quickly across the whole system. Flow control issues mean that once a receiver's buffer fills up (common when a burst of orders arrives faster than the trading engine can process them), the sender must throttle back, so acknowledgments for a whole backlog of orders get delayed together — this shows up exactly as the observed rise in average latency during the busy opening period. Nagle's algorithm interacts particularly badly with the trading platform's traffic pattern of many small, independent messages: instead of each order acknowledgment being sent immediately, it can sit in a buffer waiting to be coalesced with the next one or waiting for a pending ACK, directly adding the kind of consistent, small delay (from ~1 ms up to tens of ms) described in the case. SYN queue limitations affect the initial connection phase — if the trading platform or client applications frequently open new TCP connections (rather than reusing persistent ones), a surge of connection attempts at market open can overflow the SYN queue, causing retransmitted SYNs and added round-trip delays before any actual trade messages can even be exchanged, which explains the observed longer connection establishment times.

3. Recommend appropriate TCP configuration changes to reduce latency and improve transaction reliability.

- i. Disable Nagle's Algorithm (enable TCP_NODELAY) on sockets used for order messages, so small latency-sensitive packets are sent immediately instead of being buffered/coalesced.
- ii. Increase TCP receive and send buffer sizes (socket buffer tuning) on both the trading engine and client sides so that flow-control-induced window shrinkage is less likely under burst traffic.
- iii. Increase the SYN backlog queue size (e.g., tuning the OS-level somaxconn / tcp_max_syn_backlog parameters) to accommodate connection bursts at market open without dropping SYN's.
- iv. Use persistent, long-lived TCP connections (connection pooling/keep-alive) between clients and the trading engine instead of opening a new connection per order, avoiding repeated three-way handshake overhead entirely.
- v. Enable TCP selective acknowledgments (SACK) and tune retransmission timeout (RTO) parameters so that if loss does occur, recovery is faster and does not stall the whole connection waiting for a single retransmission.
- vi. Consider enabling explicit congestion notification (ECN) and modern congestion control algorithms (e.g., BBR) to react to congestion more precisely, avoiding unnecessary throughput cutbacks that add latency.